

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



MÔN: CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN

ĐẠI

ĐỀ TÀI: TÌM HIỂU VÀ ỨNG DỤNG SPRING BOOT

Giảng viên: Phạm Thi Vương

Nhóm 23 - Sáng thứ 7

Nhóm sinh viên thực hiện

Phạm Văn Sỹ Học - 3123560030

Trần Thế Hùng - 3123410117

Nguyễn Nhật Long - 3122410217

Huỳnh Ngọc Tuấn - 3122410446

Thành phố Hồ Chí Minh, tháng 09/2026

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

- Spring Boot hiện là framework backend phổ biến bậc nhất trong hệ sinh thái Java: nó loại bỏ phần lớn cấu hình XML rườm rà của Spring Framework truyền thống nhờ cơ chế auto-configuration và starter dependency, giúp nhóm có thể tập trung vào nghiệp vụ thay vì "đấu tranh" với cấu hình.
- Đề tài nhóm chọn để ứng dụng công nghệ là xây dựng hệ thống Backend quản lý bán sách trực tuyến — một bài toán thương mại điện tử cỡ vừa (quản lý danh mục sách, giỏ hàng, đơn hàng, tài khoản khách hàng/quản trị viên) rất phù hợp để bộc lộ đầy đủ các lớp kiến trúc điển hình của Spring Boot: Controller – Service – Repository, và các mối quan hệ dữ liệu thực tế (sách – tác giả – thể loại – đơn hàng).
- Về định hướng nghề nghiệp: khảo sát nhanh trên ITviec cho thấy có khoảng 58–60 tin tuyển dụng đang mở với từ khoá "spring boot" tại Việt Nam, chủ yếu ở các vị trí Java Backend/Fullstack Developer tại TP. Hồ Chí Minh và Hà Nội. Đây là một trong những kỹ năng backend được yêu cầu nhiều nhất đối với sinh viên Công nghệ thông tin mới ra trường theo hướng doanh nghiệp (enterprise), nên việc làm chủ Spring Boot mang lại giá trị thực tế cho hồ sơ xin việc của các thành viên trong nhóm.

2. Mục tiêu báo cáo

Mục tiêu kiến thức (các mục BẮT BUỘC LỖI — điều kiện cần)

- Hiểu và trình bày được cơ chế IoC (Inversion of Control) và Dependency Injection: bean được Spring container tạo, quản lý vòng đời và inject như thế nào, khác gì với việc tự new() đối tượng.
- Phân biệt rõ vai trò của REST Controller và tầng Service: Controller chỉ tiếp nhận request/ trả response, toàn bộ logic nghiệp vụ (tính giá, kiểm tra tồn kho sách, tạo đơn hàng...) nằm ở tầng Service.
- Sử dụng Spring Data JPA/Hibernate để thao tác với cơ sở dữ liệu quan hệ thông qua Repository, hiểu được entity, quan hệ (một sách – nhiều đánh giá, một đơn hàng – nhiều dòng chi tiết...) và câu lệnh SQL được sinh ra phía sau.

- Chuẩn hoá cấu trúc response/exception theo mã HTTP, quản lý phiên bản mã nguồn bằng Git (nhánh, pull request, review chéo), đóng gói và chạy lại được bằng Docker, quản lý bí mật cấu hình qua biến môi trường, có kiểm thử (unit test/API test), tích hợp CI (GitHub Actions) và tài liệu API (Swagger/OpenAPI).

Mục tiêu kỹ năng (các mục CHỌN THÊM / kỹ thuật nâng cao dự kiến khảo sát)

- Spring Security kết hợp JWT để phân quyền hai vai trò Khách hàng và Quản trị viên (ví dụ: chỉ Admin mới được thêm/sửa/xoá sách, chỉ chủ đơn hàng mới xem được đơn hàng của mình).
- Bean Validation cho các form nhập liệu (đăng ký tài khoản, đặt hàng) và Global Exception Handler để API trả lỗi nhất quán thay vì mỗi endpoint tự xử lý một kiểu.
- Cache (Redis/Caffeine) cho danh mục sách — dữ liệu đọc nhiều, ghi ít — nhằm giảm tải truy vấn cơ sở dữ liệu khi lượng truy cập tăng.
- Cấu hình theo môi trường (Spring Profiles: dev/test/prod) để tách biệt cấu hình khi phát triển, kiểm thử và triển khai.
- Đây là kế hoạch học tập mang tính định hướng, không phải cam kết bắt buộc đạt một mức điểm hay độ khó cố định; nhóm sẽ điều chỉnh khi hiểu bài toán sâu hơn.

3. Phạm vi nghiên cứu

Phạm vi nghiên cứu giới hạn trong 7 tuần thực hiện (theo yêu cầu rút gọn tiến độ của giảng viên hướng dẫn). Phần bắt buộc lõi tập trung vào các cơ chế nền tảng của Spring Boot đã liệt kê ở Mục tiêu kiến thức, được minh chứng bằng mã nguồn chạy được trong sản phẩm cuối — hệ thống quản lý bán sách trực tuyến gồm các module: quản lý danh mục sách/tác giả/thể loại, giỏ hàng và đặt hàng, quản lý tài khoản và phân quyền, tra cứu và tìm kiếm sách.

Phần chọn thêm dự kiến khảo sát gồm: Spring Security (đăng nhập/phân quyền Khách hàng–Quản trị viên), Bean Validation cho dữ liệu đầu vào, Global Exception Handler, Spring Profiles theo môi trường, và — nếu thời gian cho phép — Cache Redis cho danh mục sách nhằm tối ưu tốc độ truy vấn. Vì tiến độ đã được rút ngắn xuống 7 tuần, nhóm ưu tiên hoàn thành trọn vẹn phần bắt buộc lõi và 2–3 mục chọn thêm sát với bài toán (Spring Security, Bean Validation, Global Exception Handler) thay vì dàn trải; Cache Redis chỉ thực hiện nếu còn

dur thời gian ở Tuần 5. Phạm vi ban đầu này có thể được điều chỉnh trong quá trình thực hiện khi nhóm hiểu bài toán sâu hơn; mọi thay đổi sẽ được giải thích và khai báo lại trong Bảng phủ kỹ thuật ở phần cuối báo cáo.

4. Kế hoạch và tiến độ thực hiện — 7 tuần

Theo yêu cầu của giảng viên, khung tiến độ được rút gọn từ 12 tuần xuống còn 7 tuần. Các mốc dưới đây gộp lại những công việc tương ứng của khung 12 tuần ban đầu để vẫn đảm bảo đủ trọng tâm (chốt công nghệ tuần 1; hoàn thành hands-on lab và chấm chéo ở tuần cuối), áp dụng cho đề tài xây dựng Backend quản lý bán sách trực tuyến bằng Spring Boot.

Tuần	Nội dung chính	Mốc phải nộp
1	Lập nhóm, thống nhất chọn Spring Boot làm công nghệ chính, đọc Phần F của Danh mục công nghệ; cài môi trường (JDK 25+, Maven/Gradle, IntelliJ hoặc VS Code), chạy Hello World, dựng khung dự án và repo Git.	Đăng ký nhóm và công nghệ. Repo có commit thực chất của mọi thành viên.
2	Chốt bài toán đề án, thiết kế ERD và luồng nghiệp vụ (đặt hàng, thanh toán giả lập); học và làm phần BẮT BUỘC LỖI: IoC/DI, REST Controller – Service, Spring Data JPA cho entity Sách/Tác giả/Thẻ loại; viết Chương 3–4 báo cáo.	ERD/sơ đồ luồng. Bản nháp Chương 3–4 kèm code minh họa chạy được.
3	Hoàn thiện phần bắt buộc lỗi: CRUD Sách/Tác giả/Thẻ loại, chuẩn hoá response/exception theo mã HTTP.	Demo CRUD API chạy được, có test cơ bản (Postman/unit test).
4	Làm các mục CHỌN THÊM phù hợp bài toán: Spring Security + JWT (phân quyền Khách hàng–Admin), Bean Validation, Global Exception Handler.	Bảng phủ kỹ thuật bản nháp lần 1.

5	Tích hợp các kiến thức đã học vào đồ án: giỏ hàng, đơn hàng, tìm kiếm sách; refactor code Tầng 1 nếu cần; đo lường phù hợp track (độ trễ API, tỉ lệ lỗi).	Đồ án chạy được luồng chính từ đầu đến cuối. Bảng số liệu/biên bản kiểm thử.
6	Viết hands-on lab. Chấm chéo lab giữa các nhóm.	Nộp lab.
7	Hoàn thiện báo cáo, slide, video demo. Niêm phong mã nguồn.	Nộp hồ sơ đầy đủ và commit hash, trước buổi bảo vệ ít nhất 24 giờ.

PHẦN 1: TỔNG QUAN VÀ PHÂN TÍCH CÔNG NGHỆ

Mục tiêu: cung cấp bức tranh toàn cảnh và tư duy phản biện về vị thế của Spring Boot trong hệ sinh thái backend Java, làm nền tảng để nhóm lựa chọn kiến trúc phù hợp cho hệ thống quản lý bán sách trực tuyến ở Phần 3. Đây là phần chấm mã B1.

Chương 1: Giới thiệu về Spring Boot

1.1. Bối cảnh và lịch sử

Spring Boot được đội ngũ Pivotal (nay thuộc VMware/Broadcom) phát triển và phát hành phiên bản đầu tiên vào năm 2014, dựa trên nền tảng Spring Framework vốn đã ra đời từ năm 2003 do Rod Johnson khởi xướng.

Nỗi đau (pain-point) mà Spring Boot ra đời để giải quyết là sự phức tạp trong cấu hình của Spring Framework truyền thống: để chạy được một ứng dụng Spring MVC, lập trình viên phải khai báo hàng loạt file XML (web.xml, applicationContext.xml, dispatcher-servlet.xml...), tự cấu hình DataSource, ViewResolver, và cài đặt một application server ngoài (Tomcat/JBoss) để deploy file WAR. Quá trình này tốn nhiều thời gian, dễ sai sót và gây khó khăn cho người mới. Spring Boot giải quyết vấn đề này bằng ba trụ cột chính:

- Auto-configuration: tự động cấu hình các bean phổ biến (DataSource, JPA, MVC...) dựa trên các thư viện có trong classpath, giảm thiểu cấu hình thủ công.
- Starter dependencies: các gói phụ thuộc đóng gói sẵn (spring-boot-starter-web, spring-boot-starter-data-jpa...) giúp thêm một tính năng chỉ bằng một dòng khai báo dependency thay vì tự dò tìm và ghép nhiều thư viện tương thích.
- Embedded server: nhúng sẵn Tomcat/Jetty/Undertow bên trong ứng dụng, cho phép chạy bằng một lệnh (java -jar) mà không cần cài đặt server riêng — rất phù hợp với mô hình đóng gói container (Docker) sau này.

1.2. Kiến trúc và nguyên lý hoạt động

Về bản chất, Spring Boot là một framework được xây dựng trên nền Spring Framework, không phải một thư viện đơn lẻ hay một nền tảng (platform) độc lập: nó cung cấp một bộ quy ước (convention-over-configuration) và cơ chế khởi động riêng (SpringApplication.run) để nạp và khởi tạo IoC Container một cách tự động. Gọi Spring Boot là "thư viện" hoặc "nền tảng" là gọi sai loại theo phân loại framework/thư viện/nền tảng/giao thức.

- IoC Container (ApplicationContext): là trung tâm của kiến trúc - nơi các đối tượng (bean) được tạo, cấu hình và quản lý vòng đời. Khi ứng dụng khởi động, Spring quét các lớp được đánh dấu (@Component, @Service, @Repository, @RestController...) và cấu hình (@Configuration, @Bean) để xây dựng "đồ thị phụ thuộc" giữa các bean, sau đó inject chúng vào nơi cần thông qua @Autowired hoặc constructor injection.
- Auto-configuration engine: dựa vào các annotation @EnableAutoConfiguration/@SpringBootApplication và các file cấu hình có điều kiện (@ConditionalOnClass, @ConditionalOnMissingBean...), Spring Boot quyết định bean nào cần được tạo sẵn — ví dụ nếu có spring-boot-starter-data-jpa và driver MySQL trong classpath, nó tự cấu hình DataSource và EntityManagerFactory mà nhóm không cần khai báo thủ công.
- Kiến trúc phân lớp Controller – Service – Repository: đây là mô hình kiến trúc phổ biến nhất khi xây dựng REST API bằng Spring Boot, sẽ được áp dụng trực tiếp cho hệ thống bán sách - ví dụ BookController tiếp

nhận HTTP request, gọi BookService xử lý logic nghiệp vụ (kiểm tra tồn kho, tính giá sau giảm giá), và BookService gọi BookRepository (kế thừa JpaRepository) để truy vấn/ghi dữ liệu xuống cơ sở dữ liệu quan hệ thông qua Hibernate/JPA.

- Spring Data JPA & ORM: ánh xạ các entity (Book, Author, Category, Order, OrderItem, User) thành bảng dữ liệu quan hệ; các quan hệ điển hình trong bài toán bán sách gồm quan hệ một–nhiều giữa Author và Book, nhiều–nhiều giữa Book và Category, và một–nhiều giữa Order và OrderItem.

1.3. So sánh và đánh giá

Nhóm so sánh Spring Boot với hai công nghệ thường được cân nhắc thay thế khi xây dựng backend cho hệ thống thương mại điện tử cỡ vừa: NestJS (Node.js/TypeScript) và Express.js (Node.js). Phần đánh giá dưới đây dựa trên kinh nghiệm thực tế nhóm đã thử nghiệm khi dựng khung dự án ở Tuần 2 (Hello World + cấu trúc thư mục); các số liệu định lượng như thời gian khởi động, độ trễ API sẽ được nhóm đo đạc và cập nhật ở Chương 12 khi có bản build hoàn chỉnh, không phải dịch lại từ blog.

Tiêu chí	Spring Boot (Java)	NestJS (TypeScript)
Cấu hình ban đầu	Sinh dự án qua Spring Initializr, quy ước rõ ràng nhưng nhiều boilerplate (entity, DTO, mapper) cho mỗi module nghiệp vụ.	CLI nest generate tạo nhanh module/controller/service; cấu trúc gọn hơn cho dự án nhỏ và vừa.
Hệ sinh thái & ORM	Spring Data JPA/Hibernate rất trưởng thành, hỗ trợ tốt các quan hệ phức tạp (Book–Author–Category) và transaction quản lý qua @Transactional.	TypeORM/Prisma linh hoạt, cú pháp gần JavaScript thuần, nhưng hệ sinh thái plugin cho nghiệp vụ enterprise (batch, security) chưa phong phú bằng Spring.
Hiệu năng khởi động	JVM có thời gian khởi động và bộ nhớ nền cao hơn (đo cụ thể sẽ bổ sung ở Chương	Node.js khởi động nhanh hơn, phù hợp môi trường serverless/cold-start ngắn.

	12), phù hợp tiến trình chạy dài (long-running service).	
Nhu cầu tuyển dụng tại VN	~58–60 tin tuyển dụng "spring boot" đang mở trên ITviec tại thời điểm khảo sát (25/9/2026), tập trung ở vị trí Java Backend/Fullstack.	Ít tin tuyển dụng ghi rõ "NestJS" hơn so với Spring Boot trên cùng nền tảng tuyển dụng trong lần khảo sát của nhóm.

Từ bảng so sánh trên, nhóm rút ra hai trường hợp **KHÔNG NÊN** dùng Spring Boot:

- Hàm serverless/FaaS yêu cầu cold-start cực ngắn (ví dụ AWS Lambda gọi vài lần mỗi giờ): thời gian khởi tạo ApplicationContext của Spring Boot (thường vài trăm mili-giây đến vài giây tùy số lượng bean) khiến trải nghiệm cold-start kém hơn hẳn so với runtime nhẹ như Node.js hoặc các framework Java tối ưu cho GraalVM native image (Quarkus, Micronaut).
- Dịch vụ vi mô cực nhỏ, tài nguyên giới hạn nghiêm ngặt (ví dụ chạy trên thiết bị IoT hoặc container chỉ cấp 64–128MB RAM): bộ nhớ nền và số lượng thư viện mà Spring Boot nạp mặc định thường vượt quá ngân sách tài nguyên đó, trong khi các framework nhẹ hơn (Express.js, Micronaut) phù hợp hơn.

Ngược lại, với hệ thống quản lý bán sách trực tuyến — một backend chạy dài hạn, cần giao dịch dữ liệu quan hệ phức tạp (đơn hàng, tồn kho) và hưởng lợi từ hệ sinh thái Spring Security/Spring Data trưởng thành — Spring Boot là lựa chọn phù hợp và là lý do nhóm quyết định chọn công nghệ này.

Chương 2: Vị thế công nghệ và phạm vi ứng dụng

2.1. Loại sản phẩm/hệ thống thường dùng Spring Boot

Spring Boot thường được dùng để xây dựng các hệ thống backend dạng REST API hoặc microservice cho ứng dụng doanh nghiệp (enterprise), nơi cần xử lý

ng nghiệp vụ phức tạp, giao dịch dữ liệu nhất quán và khả năng mở rộng theo module. Ba nhóm ứng dụng tiêu biểu và đối tượng sử dụng chính:

- **Hệ thống thương mại điện tử/bán hàng (như đề tài của nhóm):** quản lý danh mục sản phẩm, giỏ hàng, đơn hàng, thanh toán, khách hàng và quản trị viên là hai nhóm người dùng chính.
- **Hệ thống ngân hàng/tài chính (core banking, thanh toán):** yêu cầu tính nhất quán giao dịch cao (@Transactional), bảo mật nghiêm ngặt (Spring Security, OAuth2) — nhân viên vận hành và khách hàng cuối là người dùng chính.
- **Hệ thống quản trị nội bộ doanh nghiệp (CMS, ERP, quản lý nhân sự):** phục vụ nhân viên nội bộ, tập trung vào quy trình nghiệp vụ và phân quyền theo vai trò.

2.2. Tình hình sử dụng và nhu cầu hiện tại

Theo khảo sát nhanh của nhóm trên ITviec — một trong những nền tảng tuyển dụng IT phổ biến nhất Việt Nam — vào ngày 25/9/2026, có khoảng 58–60 tin tuyển dụng đang mở với từ khoá "spring boot", trải rộng từ vị trí Junior/Middle Backend Developer đến Senior/Technical Lead/Solution Architect, tập trung chủ yếu tại TP. Hồ Chí Minh và Hà Nội, ở các lĩnh vực SaaS, fintech/ngân hàng và outsourcing cho thị trường Nhật/Úc.

Các tin tuyển dụng này thường yêu cầu kết hợp Spring Boot với hệ sinh thái mở rộng: Spring Security, Microservices, Docker/Kubernetes, message queue (Kafka/RabbitMQ), Redis/PostgreSQL, và CI/CD — cho thấy Spring Boot hiếm khi được dùng đơn lẻ mà thường là thành phần trung tâm trong một hệ sinh thái công cụ backend lớn hơn. Điều này phù hợp với định hướng của nhóm khi lựa chọn thêm các mục CHỌN THÊM (Spring Security, cấu hình theo môi trường) thay vì chỉ dừng ở CRUD cơ bản.

2.3. Xu hướng và giới hạn

Xu hướng: Spring Boot tiếp tục là lựa chọn mặc định cho backend Java trong doanh nghiệp vừa và lớn tại Việt Nam nhờ độ ổn định, tài liệu phong phú và lượng lập trình viên có sẵn trên thị trường lao động. Xu hướng gần đây là kết hợp Spring Boot với kiến trúc microservices, containerization

(Docker/Kubernetes) và các dịch vụ cloud (AWS, Azure, GCP) thay vì triển khai monolith truyền thống.

Giới hạn/ôn định ở mảng nào: với các sản phẩm cần khởi động cực nhanh (serverless, edge computing) hoặc yêu cầu footprint bộ nhớ cực nhỏ, hệ sinh thái đang dần chuyển hướng sang các framework Java tối ưu cho GraalVM native image (Quarkus, Micronaut) hoặc runtime khác (Node.js, Go).

Kết luận phạm vi **NÊN** và **KHÔNG NÊN** dùng: nhóm kết luận Spring Boot **NÊN** được dùng cho các hệ thống backend chạy dài hạn, nghiệp vụ dữ liệu quan hệ phức tạp và cần hệ sinh thái bảo mật/kiểm thử trưởng thành — đúng như bài toán quản lý bán sách trực tuyến của nhóm; và **KHÔNG NÊN** dùng cho các hàm serverless cold-start ngắn hoặc dịch vụ siêu nhẹ chạy trên thiết bị giới hạn tài nguyên, như đã phân tích ở Mục 1.3.

PHẦN 2: KIẾN THỨC BẢN CHẤT VÀ THỰC HÀNH

Mục tiêu: chứng minh nhóm hiểu công nghệ qua chuỗi Khái niệm/cơ chế → Thực hành → Kết quả → Giải thích. Chương 3 thiết lập nền tảng môi trường và kiến trúc dự án để các chương thực hành tiếp theo (Chương 4 trở đi) triển khai phần bắt buộc lõi và các mục chọn thêm trên cùng một khung dự án thống nhất.

Chương 3: Thiết lập môi trường và kiến trúc dự án

3.1. Công cụ cần thiết

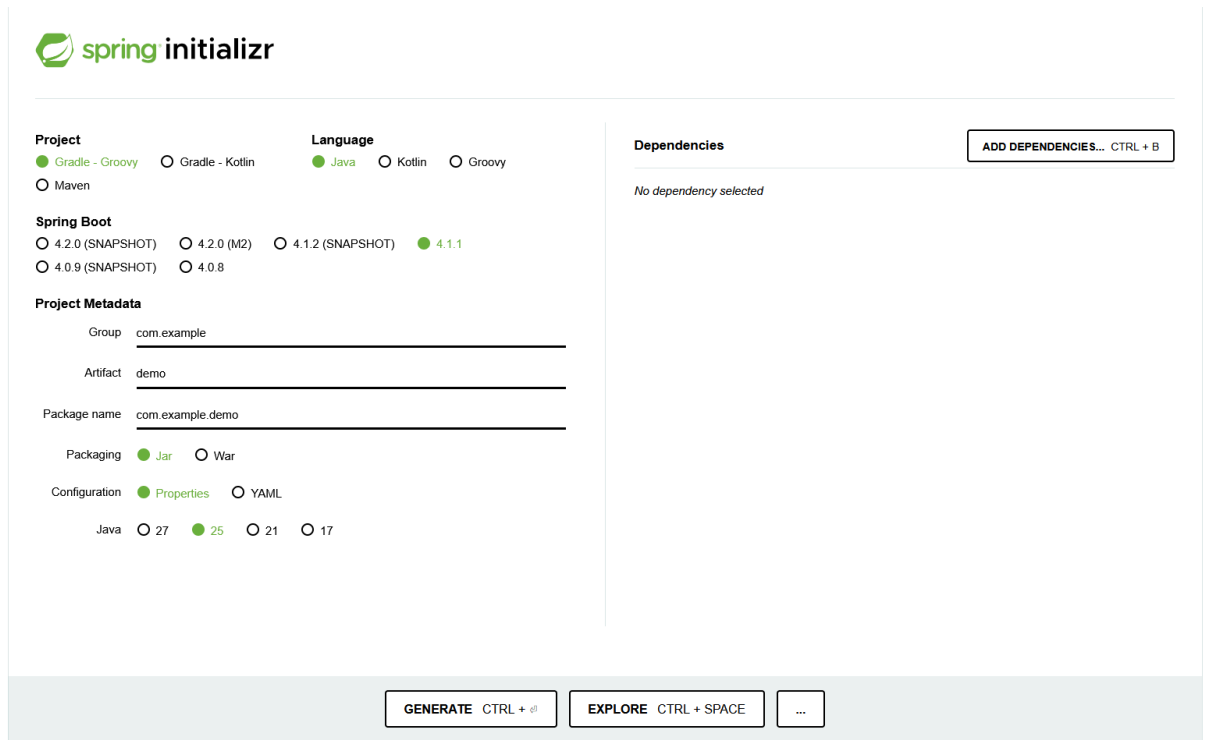
- IDE: IntelliJ IDEA (Community hoặc Ultimate) — hỗ trợ tốt Spring Boot qua plugin Spring Assistant tích hợp sẵn hoặc Visual Studio Code.
- JDK 25 (LTS) — phiên bản Java tối thiểu được khuyến nghị cho Spring Boot 4.x.
- Công cụ build: Maven (pom.xml) — nhóm chọn Maven vì cú pháp khai báo dependency quen thuộc và được đa số tài liệu Spring Boot sử dụng làm ví dụ.
- Cơ sở dữ liệu: MySQL 8 (bản cài local khi phát triển, sẽ đóng gói bằng Docker ở bước triển khai) và DBeaver để xem/kiểm tra dữ liệu.
- Docker/Docker Compose — đóng gói ứng dụng và cơ sở dữ liệu để chạy lại được bằng một lệnh trên máy khác.

- Postman (hoặc Swagger UI tích hợp sẵn qua springdoc-openapi) — để gọi thử và kiểm thử các endpoint REST API.
- Git/GitHub — quản lý phiên bản mã nguồn theo nhánh, review chéo giữa các thành viên.

3.2. Cài đặt và Hello World

Các bước cài đặt cơ bản (tóm tắt, không đi sâu từng dòng lệnh):

- Khởi tạo dự án tại Spring Initializr (start.spring.io) với: Project = Maven, Language = Java, Spring Boot 4.x, Packaging = Jar, Java 25.

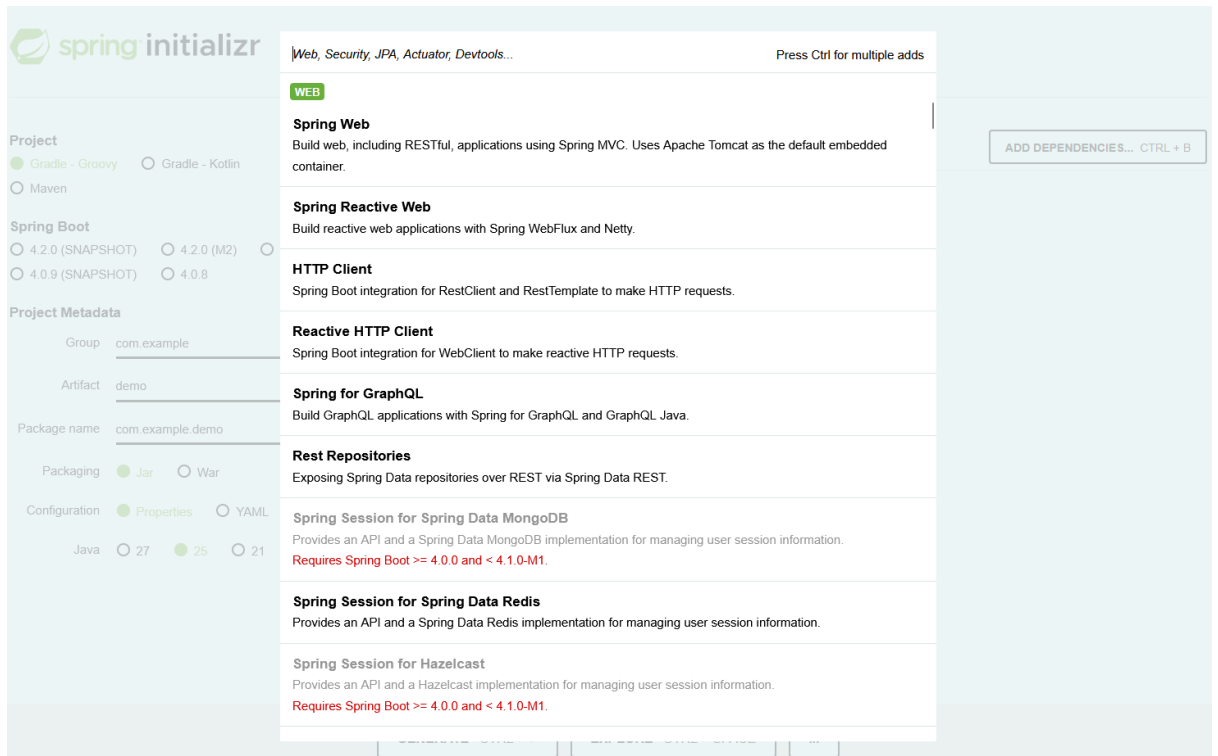


The screenshot shows the Spring Initializr web interface. The form is divided into several sections:

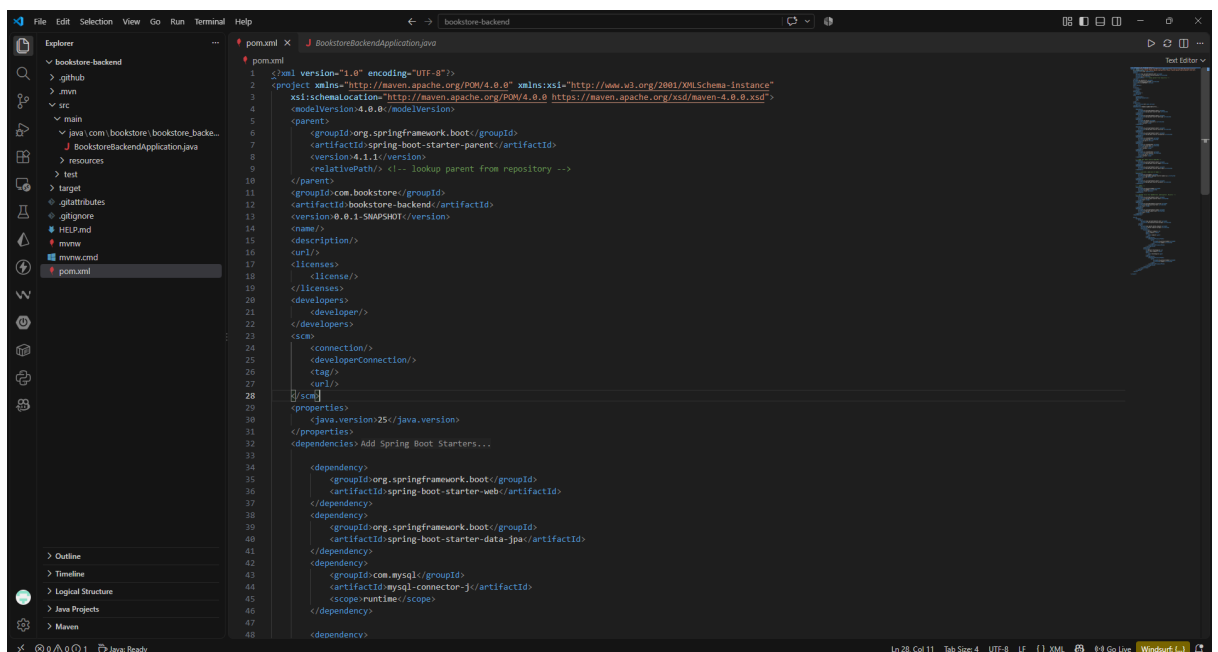
- Project:** Includes radio buttons for **Gradle - Groovy** (selected), **Gradle - Kotlin**, and **Maven**.
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for **4.2.0 (SNAPSHOT)**, **4.2.0 (M2)**, **4.1.2 (SNAPSHOT)**, **4.1.1** (selected), **4.0.9 (SNAPSHOT)**, and **4.0.8**.
- Project Metadata:** Includes text input fields for **Group** (com.example), **Artifact** (demo), and **Package name** (com.example.demo).
- Packaging:** Includes radio buttons for **Jar** (selected) and **War**.
- Configuration:** Includes radio buttons for **Properties** (selected) and **YAML**.
- Java:** Includes radio buttons for **27**, **25** (selected), **21**, and **17**.
- Dependencies:** A section on the right with a button **ADD DEPENDENCIES... CTRL + B** and the text *No dependency selected*.

At the bottom of the form, there are three buttons: **GENERATE CTRL + G**, **EXPLORE CTRL + SPACE**, and a **...** button.

- Chọn các dependency khởi điểm: Spring Web, Spring Data JPA, MySQL Driver, Validation, Lombok, springdoc-openapi (Swagger UI).



- Tải project về, mở bằng IntelliJ hoặc VS Code, chờ Maven tải xong dependency.



- Tạo một REST Controller đơn giản (ví dụ HealthController) trả về chuỗi "Hello, Spring Boot! Book Store API is running." tại đường dẫn GET /api/hello.

```

package com.bookstore.bookstore_backend.controller;

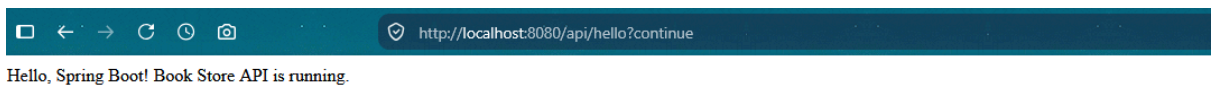
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

Windsurf: Refactor | Explain
@RestController
@RequestMapping("/api")
public class HealthController {

    Windsurf: Refactor | Explain | Generate Javadoc | X
    @GetMapping("/hello")
    public String checkHealth() {
        return "Hello, Spring Boot! Book Store API is running.";
    }
}

```

- Chạy ứng dụng bằng nút Run trong IDE hoặc lệnh `mvn spring-boot:run`, sau đó gọi thử `http://localhost:8080/api/hello` bằng trình duyệt hoặc Postman để xác nhận ứng dụng đã khởi động thành công.



http://localhost:8080/api/hello?continue

Hello, Spring Boot! Book Store API is running.

3.3. Kiến trúc

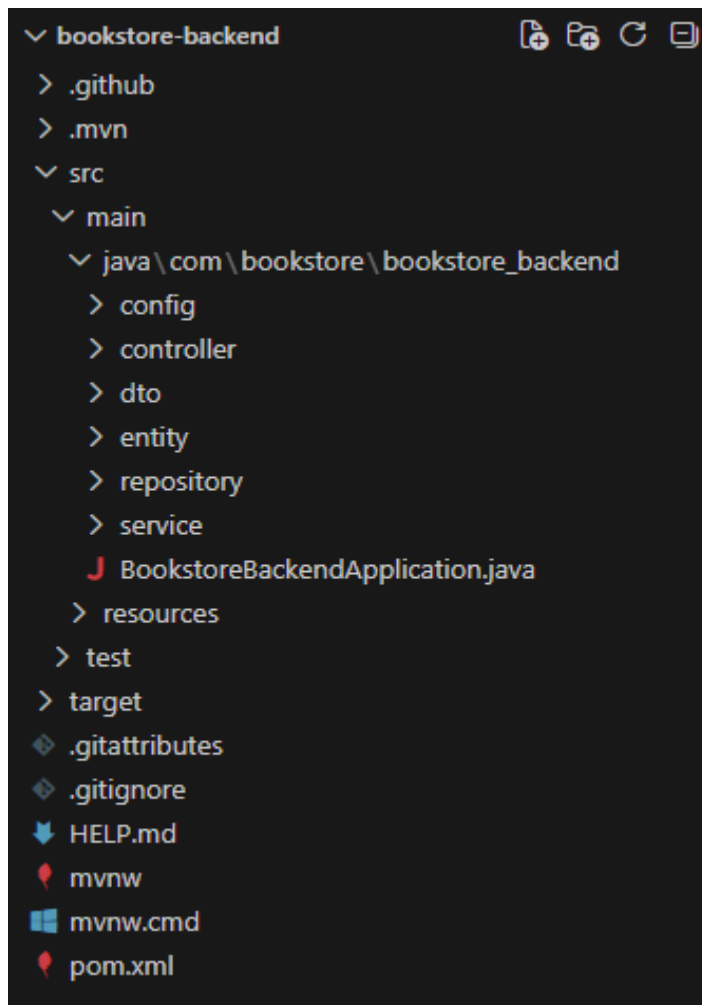
Hệ thống Backend của đồ án được thiết kế theo mô hình **kiến trúc phân lớp (Layered Architecture)** kết hợp phân chia package theo miền nghiệp vụ, nhằm đảm bảo tính đơn nhiệm (Single Responsibility), dễ bảo trì và mở rộng theo chuẩn của Spring Boot. Cấu trúc cụ thể gồm các thành phần:

- Lớp Controller (package controller): tiếp nhận HTTP request, ánh xạ DTO đầu vào, gọi Service tương ứng và trả về response chuẩn hoá (ví dụ BookController, OrderController, AuthController).

- Lớp Service (package service): chứa toàn bộ logic nghiệp vụ — kiểm tra tồn kho trước khi tạo đơn hàng, tính tổng tiền giỏ hàng, kiểm tra quyền sở hữu đơn hàng trước khi cho khách xem chi tiết.
- Lớp Repository (package repository): các interface kế thừa JpaRepository, chỉ chịu trách nhiệm truy vấn/ghi dữ liệu, không chứa logic nghiệp vụ.
- Lớp Entity/DTO (package entity, dto): Entity ánh xạ trực tiếp bảng dữ liệu (Book, Author, Category, User, Order, OrderItem); DTO dùng để giao tiếp với client, tránh lộ trực tiếp cấu trúc entity và tránh vòng lặp tham chiếu khi serialize JSON.
- Lớp Config (package config): cấu hình Spring Security, CORS, Exception Handler toàn cục và cấu hình OpenAPI/Swagger.

Luồng dữ liệu/điều khiển tổng quát: Client gửi HTTP request → DispatcherServlet của Spring MVC định tuyến đến đúng Controller theo @RequestMapping → Controller gọi Service xử lý nghiệp vụ (có thể ném exception nghiệp vụ, ví dụ BookOutOfStockException) → Service gọi Repository để đọc/ghi dữ liệu qua Hibernate xuống MySQL → kết quả được Service ánh xạ sang DTO → Controller trả JSON response kèm mã HTTP phù hợp (200, 201, 400, 404...); nếu có lỗi, GlobalExceptionHandler (@ControllerAdvice) bắt exception và trả cấu trúc lỗi thống nhất thay vì để lộ stack trace.

Cách tổ chức này phù hợp với Spring Boot vì tận dụng đúng thế mạnh của IoC/DI (mỗi lớp là một bean độc lập, dễ thay thế/kiểm thử bằng mock) và phù hợp bài toán bán sách vì mỗi domain nghiệp vụ (sách, đơn hàng, người dùng) được đóng gói riêng biệt, giúp nhóm nhiều người có thể làm song song trên các module khác nhau mà ít xung đột mã nguồn.



3.4. Thực hành khởi tạo

Minh chứng môi trường hoạt động và các thành phần chính kết nối thành công gồm:

- **Cấu hình Spring Profiles và biến môi trường:** Hệ thống sử dụng file `application.properties` để điều hướng profile mặc định và `application-prod.properties` để nạp thông tin kết nối cơ sở dữ liệu từ các biến môi trường (`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`), đảm bảo không để lộ mật khẩu trong mã nguồn.
- **Đóng gói dịch vụ với Docker Compose:** Hệ thống khai báo hai service gồm `mysql-db` (chạy MySQL 8 kèm volume nạp dữ liệu mẫu `init.sql`) và `book-store-api` (được build từ Dockerfile với môi trường OpenJDK 25), cho phép triển khai toàn bộ hệ thống bằng một lệnh duy nhất.

```

version: '3.8'

services:
  > Run All Services
  > Run Service
  mysql-db:
    image: mysql:8.0
    container_name: mysql-db
    restart: always
    environment:
      MYSQL_DATABASE: bookstore
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  > Run Service
  book-store-api:
    build: .
    container_name: book-store-api
    restart: always
    depends_on:
      - mysql-db
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_URL: jdbc:mysql://mysql-db:3306/bookstore?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Ho_Chi_Minh
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}

volumes:
  mysql_data:

```

- **Kiểm tra trạng thái kết nối:** Sử dụng endpoint `/actuator/health` và log khởi động của Hibernate để xác nhận ứng dụng đã tạo kết nối thành công tới cơ sở dữ liệu trước khi đi vào xây dựng các tầng nghiệp vụ chi tiết.
- **Tích hợp liên tục với GitHub Actions (Pipeline mẫu):** Thiết lập file workflow `.github/workflows/ci.yml` để tự động kích hoạt tiến trình cài đặt JDK 25, kiểm tra định dạng và chạy lệnh `mvn clean verify` (chạy test tự động) mỗi khi có commit hoặc Pull Request được đẩy lên GitHub.


```
name: Java CI with Maven

on:
  push:
    branches: [ "main", "dev" ]
  pull_request:
    branches: [ "main", "dev" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          java-version: '25'
          distribution: 'temurin'
          cache: maven

      - name: Build and Test with Maven
        run: mvn -B package
```